Zephyr Project

IT-Schulungen.com
Technology & Transformation

Kurzcheck Thread-Grundlagen

Zuordnen, entscheiden, begründen

2.04

Kurzcheck in drei Schritten

01	`Zuordnen`	Stack · Threadobjekt · Entry · Priorität · Startzeit
02	`Entscheiden`	statisch oder dynamisch · sofort oder verzögert
03	`Begründen`	Lebensdauer und vorhergesagtes Startverhalten erklären

Arbeitszeit: 60 Sekunden pro Codekarte – erst markieren, dann begründen.

Codekarte A: statischer Thread

CODEKARTE A

```
K_THREAD_DEFINE(sensor_tid,
                1024,
                sensor_worker,
                NULL, NULL, NULL,
                5,
                0,
                K_MSEC(200));
```

01 Zuordnen

Markiere Name, Stackgröße, Entry-Funktion, Priorität und Startzeit.
Welche Bestandteile erzeugt das Makro statisch?

02 Vorhersagen

Wann wird sensor_worker erstmals bereit?
Welche Änderung würde den Thread sofort starten?

Codekarte B: dynamischer Thread

CODEKARTE B

```
K_THREAD_STACK_DEFINE(worker_stack, 1024);
static struct k_thread worker_data;

k_tid_t tid = k_thread_create(
    &worker_data, worker_stack,
    K_THREAD_STACK_SIZEOF(worker_stack),
    sensor_worker, NULL, NULL, NULL,
    5, 0, K_FOREVER);

k_thread_start(tid);
```

01 Stack

Welche Deklaration stellt den Speicher bereit?

02 Threadobjekt

Welches Objekt muss den Thread überleben?

03 Start

Warum läuft der Thread erst nach k_thread_start()?

Auswertung und Transfer

ZUORDNUNG

Codekarte B

Stack: worker_stack

Threadobjekt: worker_data

Entry: sensor_worker

Priorität: 5

START

Vorhersage

K_MSEC(200): verzögert

K_NO_WAIT: sofort

K_FOREVER: nicht automatisch

k_thread_start(): macht bereit

LEBENSDAUER

Klare Regel

Stack bleibt gültig

Threadobjekt bleibt gültig

bis der Thread beendet ist

Offene Frage vor dem Scheduling-Block: Welcher bereite Thread läuft als Nächstes – und warum?